

Django admin.py — FULL WOW-конспект

1. Что такое admin.py

Суть

Файл admin.py настраивает встроенную админку Django: какие модели показывать в /admin и как именно их там отображать.

ВАЖНО

Это не бизнес-логика и не описание таблиц БД. Структуру данных задают models.py и миграции.

2. Базовая идея на примере

admin.py

```
from django.contrib import admin

@admin.register(Currency)
class CurrencyAdmin(admin.ModelAdmin):
    list_display = ("code", "name", "is_active", "created_at")
```

Как это читать

Есть модель Currency — это сами данные.

Есть класс CurrencyAdmin — это настройки экрана админки для этой модели.

3. Основные инструменты admin.py

list_display

Какие колонки показывать в списке записей.

Пример: код, название, активность, дата создания.

search_fields

По каким полям работает поиск вверху списка.

Пример: искать валюту по коду USD или по названию.

list_filter

Какие быстрые фильтры показывать справа в админке.

Пример: активные / неактивные записи.

`autocomplete_fields`

Умный поиск по связанным сущностям вместо длинного выпадающего списка.
Особенно полезно для `ForeignKey`.

4. Что значит двойное подчёркивание `__`

Поиск и связи в Django ORM

```
currency__code
source__name
created_by__username
uploaded_by__username
```

Смысл

Запись вида `currency__code` означает: взять связанную модель `Currency` и обратиться к её полю `code`.

ВАЖНО

Оператор `__` — это базовая идея Django ORM. Он потом будет встречаться в фильтрах, запросах, сортировках и аннотациях.

5. Что увидит администратор в реальной админке

`CurrencyAdmin`

Справочник валют: код, название, активность, дата создания.
Можно быстро найти USD и отфильтровать активные записи.

`ExchangeRateAdmin`

Журнал курсов: валюта, дата курса, курс, источник, ручная корректировка.
Это уже почти рабочий журнал курсов валют.

`CommentAdmin`

Комментарии: ID, автор, активность, дата создания.
Сам текст часто не выводят в список, чтобы таблица не становилась неудобной.

`AttachmentAdmin`

Вложения: имя файла, кто загрузил, размер, тип, активность, дата.
Удобно для диагностики и ручной проверки файлов.

AuditLogAdmin

Журнал аудита: тип сущности, ID сущности, действие, кто сделал, когда сделал.
Полезно для расследования проблем и контроля изменений.

6. Как admin.py связан с остальным проектом

Цепочка

1. models.py описывает данные.
2. migrations создают / меняют таблицы БД.
3. admin.py настраивает удобный интерфейс поверх этих моделей.
4. В /admin появляется служебный UI для разработчика или администратора.

ВАЖНО

Админка — это не интерфейс для конечного пользователя. Это служебный инструмент для ввода, проверки, редактирования и диагностики данных.

7. Короткая аналогия

Простыми словами

models.py — структура таблиц и связей.

admin.py — настройка формы, списка и фильтров для администратора.

То есть: models.py = данные, admin.py = управление данными.

8. Что важно запомнить новичку

Главное

- admin.py не создаёт таблицы БД.
- Он не хранит данные.
- Он делает админку удобной: колонки, поиск, фильтры, выбор связей.
- Если модель не зарегистрирована в admin.py, в /admin её обычно не видно.

Итоговая схема

```
# Ментальная модель
models.py  -> что хранится
admin.py   -> как этим управлять в /admin
views.py   -> как с этим работает веб-логика
```